

AI Assisted Coding

Assignment -7.1

B.Tejaswi

2303A52123

Batch-40

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print "Hello"`). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

```
def greet():
```

```
print "Hello, AI Debugging Lab!"
```

```
greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1:

- Corrected code with proper syntax and AI explanation.

Prompt:

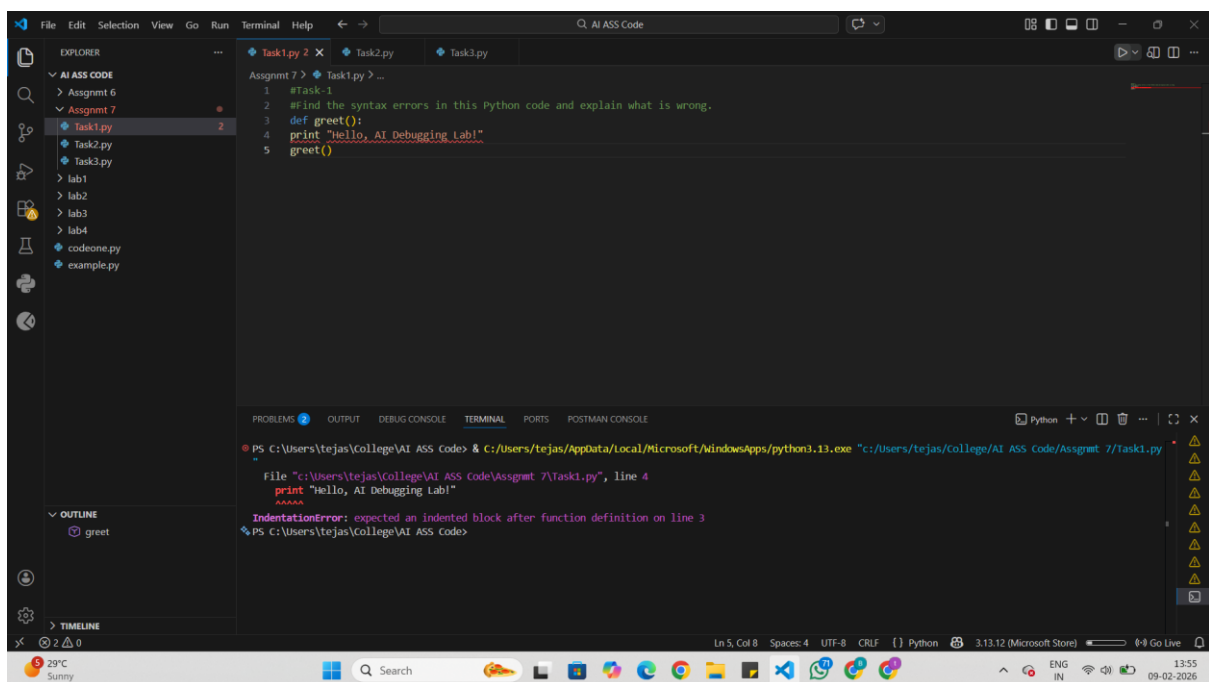
#Find the syntax errors in this Python code and explain what is wrong.

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.

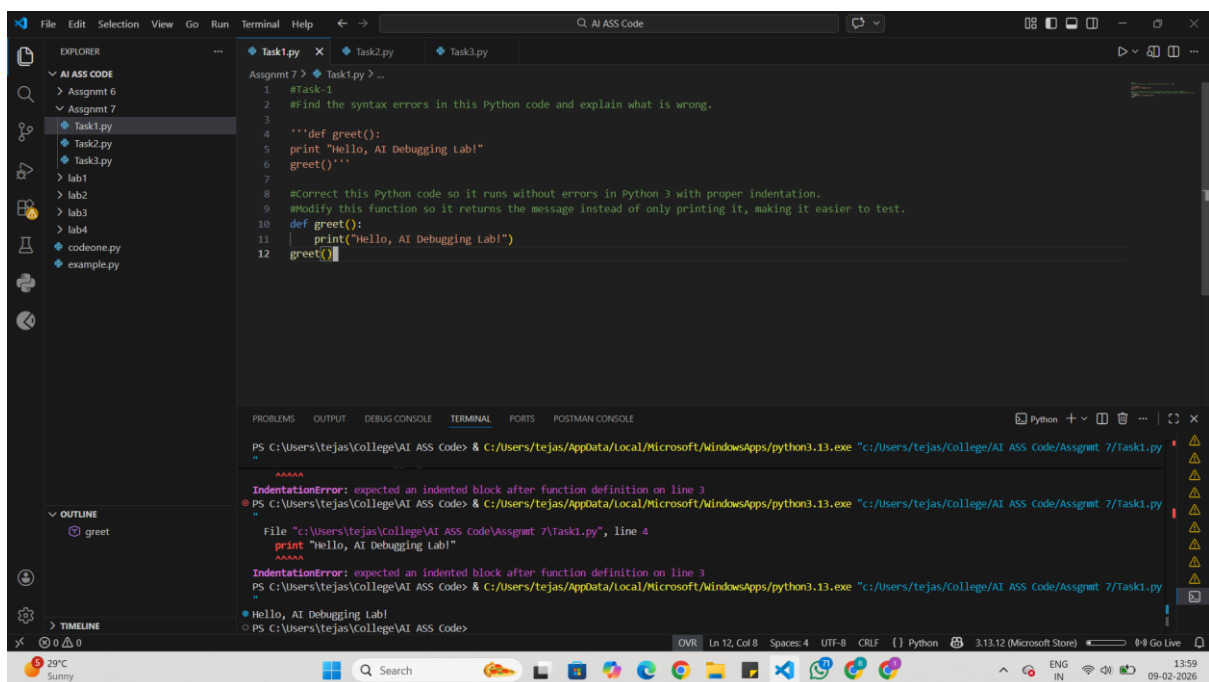


The screenshot shows the Visual Studio Code editor with a file named `Task1.py` open. The code in the editor is as follows:

```
1 #Task:1
2 #Find the syntax errors in this Python code and explain what is wrong.
3 def greet():
4 print "Hello, AI Debugging Lab!"
5 greet()
```

The terminal window at the bottom shows the command `python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assgmt 7/Task1.py"` and the resulting error:

```
File "c:/Users/tejas/College/AI ASS Code/Assgmt 7/Task1.py", line 4
    print "Hello, AI Debugging Lab!"
    ^^^^^
IndentationError: expected an indented block after function definition on line 3
```

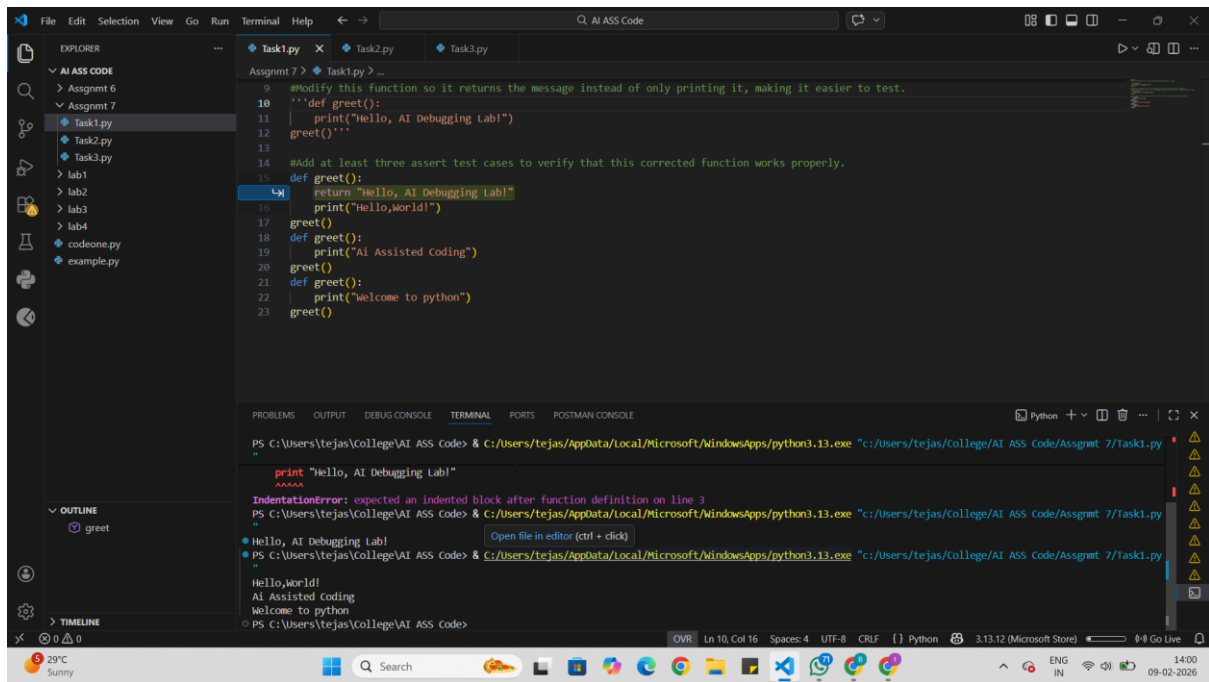


The screenshot shows the Visual Studio Code editor with the corrected code in `Task1.py`:

```
1 #Task:1
2 #Find the syntax errors in this Python code and explain what is wrong.
3
4 '''def greet():
5     print "Hello, AI Debugging Lab!"
6     greet()'''
7
8 #Correct this Python code so it runs without errors in Python 3 with proper indentation.
9 #Modify this function so it returns the message instead of only printing it, making it easier to test.
10 def greet():
11     print("Hello, AI Debugging Lab!")
12     return greet()
```

The terminal window shows the command `python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assgmt 7/Task1.py"` and the output:

```
PS C:\Users\tejas\College\AI ASS Code> python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assgmt 7/Task1.py"
Hello, AI Debugging Lab!
```



Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):
```

```
    if n = 10:
```

```
        return "Ten"
```

```
    else:
```

```
        return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test execution.

Prompt:

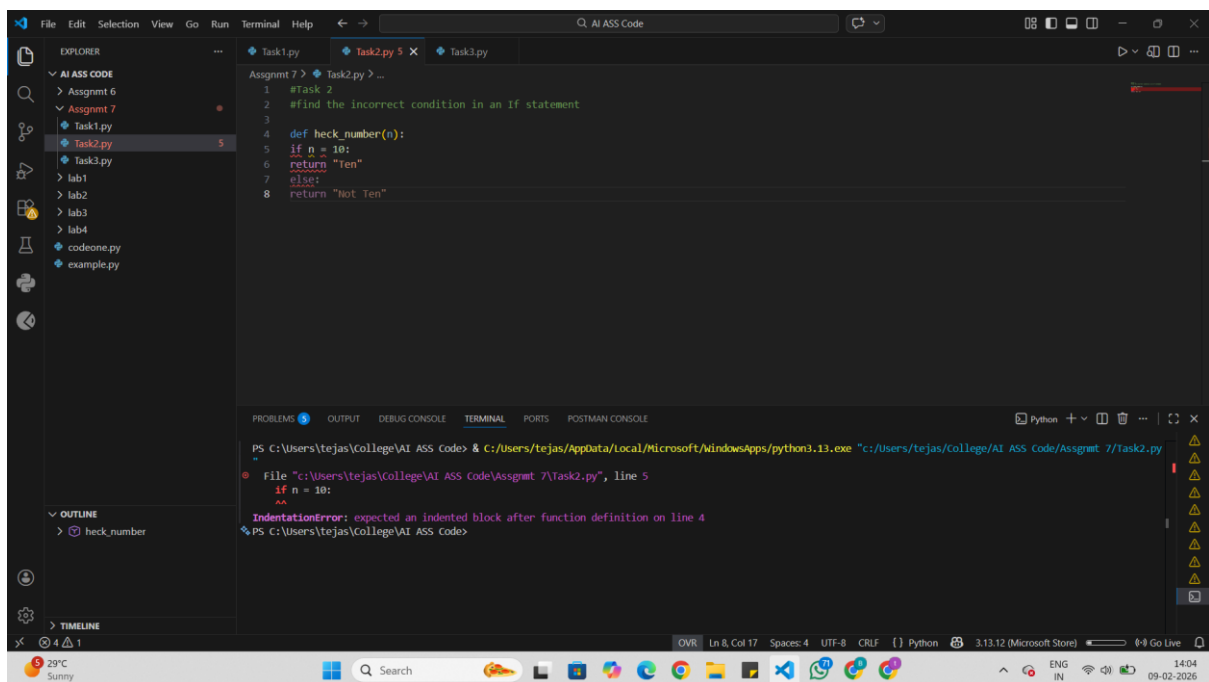
find the incorrect condition in an If statement.

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.



The screenshot shows the Visual Studio Code editor with a Python file named `Task2.py` open. The code defines a function `check_number(n)` that returns "Ten" if `n` is 10, and "Not Ten" otherwise. It then prompts the user to enter a number and prints the result. The terminal at the bottom shows the command to run the script, the user input "10", and the output "Ten".

```
1 #Task 2
2 #find the incorrect condition in an If statement
3
4 '''def check_number(n):
5     if n= 10:
6         return "Ten"
7     else:
8         return "Not Ten"'''
9
10 #correct this Python code so it runs without errors in Python 3 with proper indentation and give dynamic input to the function.
11 #modify this function so it returns the message instead of only printing it, making it easier to test
12 def check_number(n):
13     if n == 10:
14         return "Ten"
15     else:
16         return "Not Ten"
17     # Dynamic input from user
18     num = int(input("Enter a number: "))
19     result = check_number(num)
20     print(result)
21
```

Terminal output:

```
PS C:\Users\tejas\College\AI ASS Code> C:/Users/tejas/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assignmt 7/Task2.py"
Enter a number: 10
Ten
PS C:\Users\tejas\College\AI ASS Code>
```

The screenshot shows the same `Task2.py` file, but now with three unit tests added at the end of the script. The terminal shows the script being run multiple times, each time with a different input (10, 5, 0) and the output "All test cases passed!".

```
22 #Add at least three assert test cases to verify that this corrected function works properly.
23 assert check_number(10) == "Ten"
24 assert check_number(5) == "Not Ten"
25 assert check_number(0) == "Not Ten"
26 print("All test cases passed!")
```

Terminal output:

```
PS C:\Users\tejas\College\AI ASS Code> C:/Users/tejas/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assignmt 7/Task2.py"
All test cases passed!
PS C:\Users\tejas\College\AI ASS Code> C:/Users/tejas/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assignmt 7/Task2.py"
Enter a number: 5
Not Ten
All test cases passed!
PS C:\Users\tejas\College\AI ASS Code> C:/Users/tejas/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assignmt 7/Task2.py"
Enter a number: 5
Not Ten
All test cases passed!
PS C:\Users\tejas\College\AI ASS Code>
```

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()  
print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

Prompts:

Runtime Error – File Not Found

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.

The screenshot shows the Visual Studio Code editor with a Python file named `Task3.py`. The code is as follows:

```
1 #task3
2 #AI Prompt Used for Runtime Error Debugging".
3 def read_file(filename):
4     with open(filename, 'r') as f: import filename
5     return f.read()
6 print(read_file("nonexistent.txt"))
```

The error message in the terminal is:

```
File "c:\Users\tejas\College\AI ASS Code\Assgmt 7\Task3.py", line 4
    with open(filename, 'r') as f: import filename
    ^^^^^
IndentationError: expected an indented block after function definition on line 3
PS C:\Users\tejas\College\AI ASS Code>
```

The screenshot shows the Visual Studio Code editor with a Python file named `Task3.py`. The code is as follows:

```
1 # Task Description #3
2 # Runtime Error - File Not Found
3
4 def read_file(filename):
5     if not filename:
6         return "Error: Invalid file path."
7     try:
8         with open(filename, 'r') as f:
9             return f.read()
10    except FileNotFoundError:
11        return "Error: File not found."
12    except OSError:
13        return "Error: Invalid file path."
14
15 # ----- Demonstration -----
16 print(read_file("nonexistent.txt"))
17
18 # ----- Test Cases -----
19
20 # Test Case 1: File is missing
21 assert read_file("nonexistent.txt") == "Error: File not found."
22
23 # Test Case 2: File exists
24 with open("sample.txt", "w") as f:
25     f.write("AI Debugging Lab")
26
27 assert read_file("sample.txt") == "AI Debugging Lab"
```

The error message in the terminal is:

```
AssertionError
PS C:\Users\tejas\College\AI ASS Code> & C:\Users\tejas\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:\Users\tejas\College\AI ASS Code\Assgmt 7\Task3.py"
Error: File not found.
All test cases passed!
PS C:\Users\tejas\College\AI ASS Code>
```

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

class Car:

```
def start(self):  
    return "Car started"  
  
my_car = Car()  
print(my_car.drive()) # drive() is not defined
```

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.

Prompts:

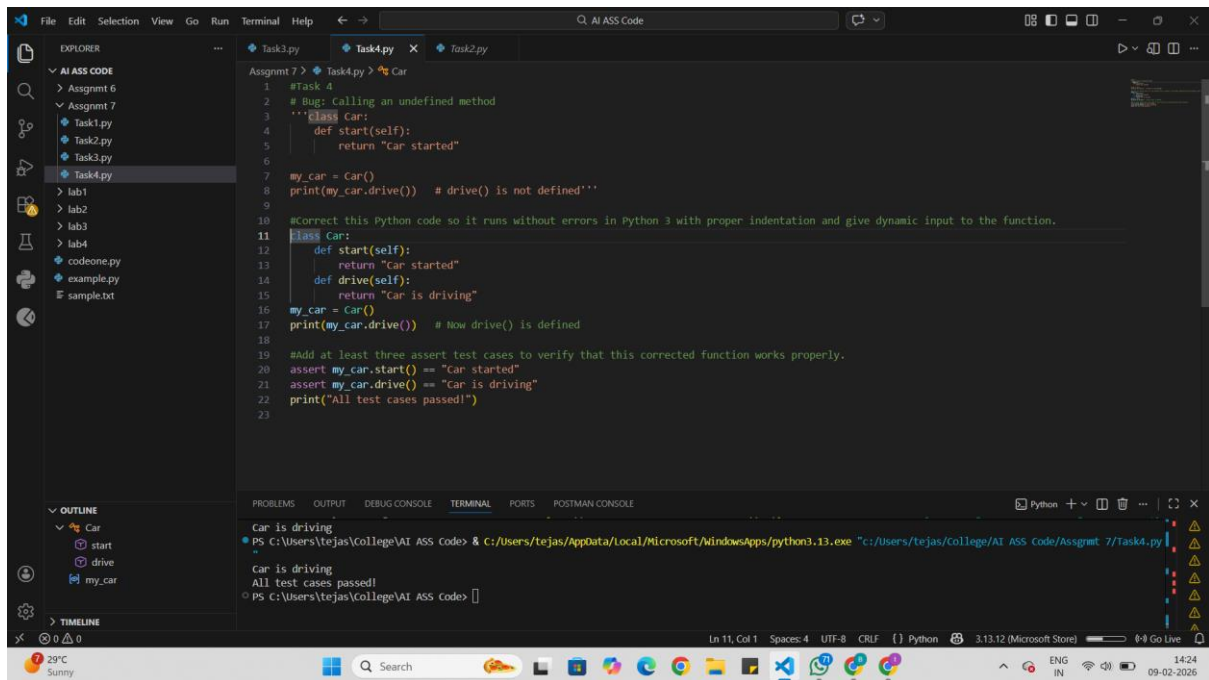
Bug: Calling an undefined method

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.



Task Description #5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```

def add_five(value):
    return value + 5
print(add_five("10"))

```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Prompts:

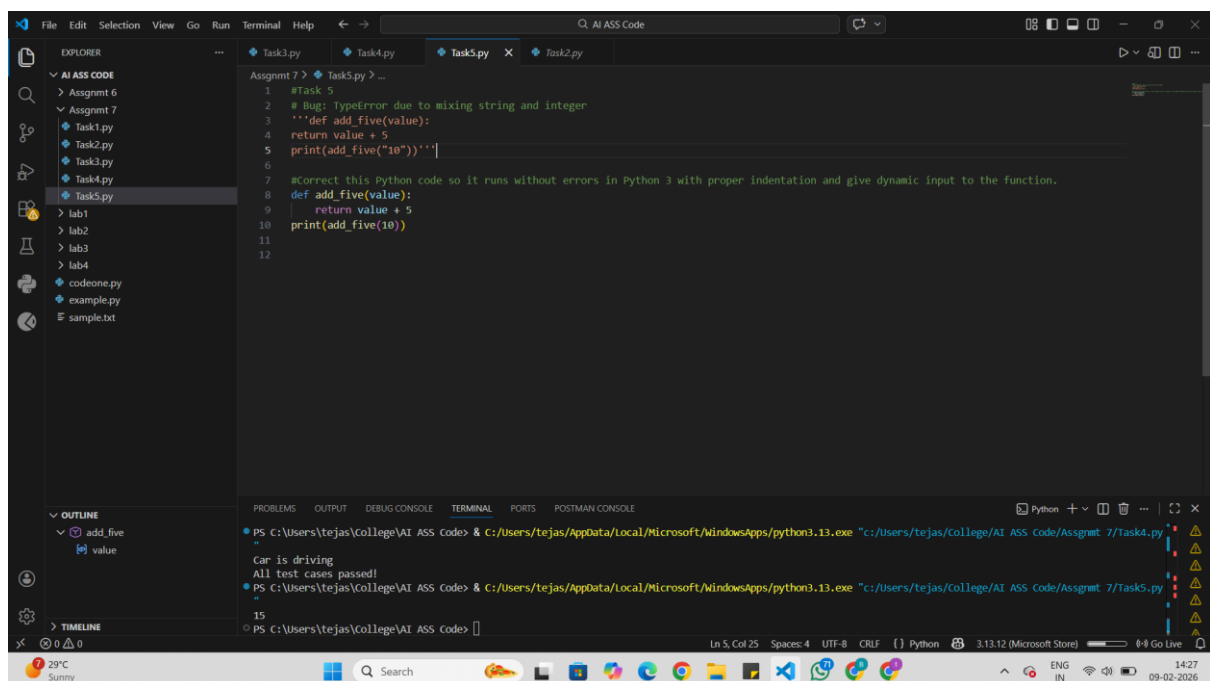
Bug: TypeError due to mixing string and integer

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.



The screenshot shows a Visual Studio Code editor with a file named `Task5.py` open. The code in the file is as follows:

```
1 #Task 5
2 # Bug: TypeError due to mixing string and integer
3 '''def add_five(value):
4     return value + 5
5     print(add_five("10"))'''
6
7 #Correct this Python code so it runs without errors in Python 3 with proper indentation and give dynamic input to the function.
8 def add_five(value):
9     return value + 5
10    print(add_five(10))
11
12
```

The terminal window at the bottom shows the execution of the code:

```
PS C:\Users\tejas\College\AI ASS Code> & C:/Users/tejas/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assignmt 7/Task4.py"
Car is driving
All test cases passed!
PS C:\Users\tejas\College\AI ASS Code> & C:/Users/tejas/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/tejas/College/AI ASS Code/Assignmt 7/Task5.py"
15
PS C:\Users\tejas\College\AI ASS Code>
```

